V
ARCHITECTURE

15
WHAT IS ARCHITECTURE?

The word “architecture” conjures visions of power and mystery. It makes us think of weighty decisions and deep technical prowess. Software architecture is at the pinnacle of technical achievement. When we think of a software architect, we think of someone who has power, and who commands respect. What young aspiring software developer has not dreamed of one day becoming a software architect?

But what is software architecture? What does a software architect do, and when does he or she do it?

First of all, a software architect is a programmer; and continues to be a programmer. Never fall for the lie that suggests that software architects pull back from code to focus on higher-level issues. They do not! Software architects are the best programmers, and they continue to take programming tasks, while they also guide

the rest of the team toward a design that maximizes productivity. Software architects may not write as much code as other programmers do, but they continue to engage in programming tasks. They do this because they cannot do their jobs properly if they are not experiencing the problems that they are creating for the rest of the programmers.

The architecture of a software system is the shape given to that system by those who build it. The form of that shape is in the division of that system into components, the arrangement of those components, and the ways in which those components communicate with each other.

The purpose of that shape is to facilitate the development, deployment, operation, and maintenance of the software system contained within it.

> *The strategy behind that facilitation is to leave as many options open as possible, for as long as possible.*

Perhaps this statement has surprised you. Perhaps you thought that the goal of software architecture was to make the system work properly. Certainly we want the system to work properly, and certainly the architecture of the system must support that as one of its highest priorities.

However, the architecture of a system has very little bearing on whether that system works. There are many systems out there, with terrible architectures, that work just fine. Their troubles do not lie in their operation; rather, they occur in their deployment, maintenance, and ongoing development.

This is not to say that architecture plays no role in supporting the proper behavior of the system. It certainly does, and that role is critical. But the role is passive and cosmetic, not active or essential. There are few, if any, *behavioral* options that the architecture of a system can leave open.

The primary purpose of architecture is to support the life cycle of the system. Good architecture makes the system easy to understand, easy to develop, easy to maintain, and easy to deploy. The ultimate goal is to minimize the lifetime cost of the system and to maximize programmer productivity.

DEVELOPMENT

A software system that is hard to develop is not likely to have a long and healthy

lifetime. So the architecture of a system should make that system easy to develop, for the team(s) who develop it.

Different team structures imply different architectural decisions. On the one hand, a small team of five developers can quite effectively work together to develop a monolithic system without well-defined components or interfaces. In fact, such a team would likely find the strictures of an architecture something of an impediment during the early days of development. This is likely the reason why so many systems lack good architecture: They were begun with none, because the team was small and did not want the impediment of a superstructure.

On the other hand, a system being developed by five different teams, each of which includes seven developers, cannot make progress unless the system is divided into well-defined components with reliably stable interfaces. If no other factors are considered, the architecture of that system will likely evolve into five components—one for each team.

Such a component-per-team architecture is not likely to be the best architecture for deployment, operation, and maintenance of the system. Nevertheless, it is the architecture that a group of teams will gravitate toward if they are driven solely by development schedule.

DEPLOYMENT

To be effective, a software system must be deployable. The higher the cost of deployment, the less useful the system is. A goal of a software architecture, then, should be to make a system that can be easily deployed *with a single action*.

Unfortunately, deployment strategy is seldom considered during initial development. This leads to architectures that may make the system easy to develop, but leave it very difficult to deploy.

For example, in the early development of a system, the developers may decide to use a “micro-service architecture.” They may find that this approach makes the system very easy to develop since the component boundaries are very firm and the interfaces relatively stable. However, when it comes time to deploy the system, they may discover that the number of micro-services has become daunting; configuring the connections between them, and the timing of their initiation, may also turn out to be a huge source of errors.

Had the architects considered deployment issues early on, they might have decided on fewer services, a hybrid of services and in-process components, and a more integrated means of managing the interconnections.

OPERATION

The impact of architecture on system operation tends to be less dramatic than the impact of architecture on development, deployment, and maintenance. Almost any operational difficulty can be resolved by throwing more hardware at the system without drastically impacting the software architecture.

Indeed, we have seen this happen over and over again. Software systems that have inefficient architectures can often be made to work effectively simply by adding more storage and more servers. The fact that hardware is cheap and people are expensive means that architectures that impede operation are not as costly as architectures that impede development, deployment, and maintenance.

This is not to say that an architecture that is well tuned to the operation of the system is not desirable. It is! It's just that the cost equation leans more toward development, deployment, and maintenance.

Having said that, there is another role that architecture plays in the operation of the system: A good software architecture communicates the operational needs of the system.

Perhaps a better way to say this is that the architecture of a system makes the operation of the system readily apparent to the developers. Architecture should reveal operation. The architecture of the system should elevate the use cases, the features, and the required behaviors of the system to first-class entities that are visible landmarks for the developers. This simplifies the understanding of the system and, therefore, greatly aids in development and maintenance.

MAINTENANCE

Of all the aspects of a software system, maintenance is the most costly. The never-ending parade of new features and the inevitable trail of defects and corrections consume vast amounts of human resources.

The primary cost of maintenance is in *spelunking* and risk. Spelunking is the cost of digging through the existing software, trying to determine the best place and the best strategy to add a new feature or to repair a defect. While making such changes, the likelihood of creating inadvertent defects is always there, adding to the cost of risk.

A carefully thought-through architecture vastly mitigates these costs. By separating the system into components, and isolating those components through stable interfaces, it is possible to illuminate the pathways for future features and greatly reduce the risk of inadvertent breakage.

KEEPING OPTIONS OPEN

As we described in an earlier chapter, software has two types of value: the value of its behavior and the value of its structure. The second of these is the greater of the two because it is this value that makes software *soft*.

Software was invented because we needed a way to quickly and easily change the behavior of machines. But that flexibility depends critically on the shape of the system, the arrangement of its components, and the way those components are interconnected.

The way you keep software soft is to leave as many options open as possible, for as long as possible. What are the options that we need to leave open? *They are the details that don't matter.*

All software systems can be decomposed into two major elements: policy and details. The policy element embodies all the business rules and procedures. The policy is where the true value of the system lives.

The details are those things that are necessary to enable humans, other systems, and programmers to communicate with the policy, but that do not impact the behavior of the policy at all. They include IO devices, databases, web systems, servers, frameworks, communication protocols, and so forth.

The goal of the architect is to create a shape for the system that recognizes policy as the most essential element of the system while making the details *irrelevant* to that policy. This allows decisions about those details to be *delayed* and *deferred*.

For example:

- It is not necessary to choose a database system in the early days of development, because the high-level policy should not care which kind of database will be used. Indeed, if the architect is careful, the high-level policy will not care if the database is relational, distributed, hierarchical, or just plain flat files.
- It is not necessary to choose a web server early in development, because the high-level policy should not know that it is being delivered over the web. If the high-level policy is unaware of HTML, AJAX, JSP, JSF, or any of the rest of the alphabet soup of web development, then you don't need to decide which web system to use until much later in the project. Indeed, *you don't even have to decide if the system will be delivered over the web*.
- It is not necessary to adopt REST early in development, because the high-level policy should be agnostic about the interface to the outside world. Nor is it necessary to adopt a micro-services framework, or a SOA framework. Again, the high-level policy should not care about these things.
- It is not necessary to adopt a dependency injection framework early in development, because the high-level policy should not care how dependencies are resolved.

I think you get the point. If you can develop the high-level policy without committing to the details that surround it, you can delay and defer decisions about those details for a long time. And the longer you wait to make those decisions, *the more information you have with which to make them properly*.

This also leaves you the option to try different experiments. If you have a portion of the high-level policy working, and it is agnostic about the database, you could try connecting it to several different databases to check applicability and performance. The same is true with web systems, web frameworks, or even the web itself.

The longer you leave options open, the more experiments you can run, the more things you can try, and the more information you will have when you reach the point at which those decisions can no longer be deferred.

What if the decisions have already been made by someone else? What if your company has made a commitment to a certain database, or a certain web server, or a certain framework? *A good architect pretends that the decision has not been made*, and shapes the system such that those decisions can still be deferred or changed for as long as possible.

> *A good architect maximizes the number of decisions not made.*

DEVICE INDEPENDENCE

As an example of this kind of thinking, let's take a trip back to the 1960s, when computers were teenagers and most programmers were mathematicians or engineers from other disciplines (and-one third or more were women).

In those days we made a lot of mistakes. We didn't know they were mistakes at the time, of course. How could we?

One of those mistakes was to bind our code directly to the IO devices. If we needed to print something on a printer, we wrote code that used the IO instructions that would control the printer. Our code was *device dependent*.

For example, when I wrote PDP-8 programs that printed on the teleprinter, I used a set of machine instructions that looked like this:

Click here to view code image

```
PRTCHR, 0
        TSF
        JMP .-1
        TLS
        JMP I PRTCHR
```

`PRTCHR` is a subroutine that prints one character on the teleprinter. The beginning zero was used as the storage for the return address. (Don't ask.) The `TSF` instruction skipped the next instruction if the teleprinter was ready to print a character. If the teleprinter was busy, then `TSF` just fell through to the `JMP .-1` instruction, which just jumped back to the `TSF` instruction. If the teleprinter was ready, then `TSF` would skip to the `TLS` instruction, which sent the character in the `A` register to the teleprinter. Then the `JMP I PRTCHR` instruction returned to the caller.

At first this strategy worked fine. If we needed to read cards from the card reader, we used code that talked directly to the card reader. If we needed to punch cards, we wrote code that directly manipulated the punch. The programs worked perfectly. How could we know this was a mistake?

But big batches of punched cards are difficult to manage. They can be lost, mutilated, spindled, shuffled, or dropped. Individual cards can be lost and extra cards can be inserted. So data integrity became a significant problem.

Magnetic tape was the solution. We could move the card images to tape. If you drop a magnetic tape, the records don't get shuffled. You can't accidentally lose a record, or insert a blank record simply by handing the tape. The tape is much more secure. It's also faster to read and write, and it is very easy to make backup copies.

Unfortunately, all our software was written to manipulate card readers and card punches. Those programs had to be rewritten to use magnetic tape. That was a big job.

By the late 1960s, we had learned our lesson—and we invented *device independence*. The operating systems of the day abstracted the IO devices into software functions that handled unit records that looked like cards. The programs would invoke operating system services that dealt with abstract unit-record devices. Operators could tell the operating system whether those abstract services should be connected to card readers, magnetic tape, or any other unit-record device.

Now the same program could read and write cards, or read and write tape, *without any change*. The Open–Closed Principle was born (but not yet named).

JUNK MAIL

In the late 1960s, I worked for a company that printed junk mail for clients. The clients would send us magnetic tapes with unit records containing the names and addresses of their customers, and we would write programs that printed nice personalized advertisements.

You know the kind:

Hello Mr. Martin,

Congratulations!

We chose YOU from everyone else who lives on Witchwood Lane to participate in our new fantastic one-time-only offering…

The clients would send us huge rolls of form letters with all the words except the name and address, and any other element they wanted us to print. We wrote programs that extracted the names, addresses, and other elements from the magnetic tape, and printed those elements exactly where they needed to appear on the forms.

These rolls of form letters weighed 500 pounds and contained thousands of letters.

Clients would send us hundreds of these rolls. We would print each one individually.

At first, we had an IBM 360 doing the printing on its sole line printer. We could print a few thousand letters per shift. Unfortunately, this tied up a very expensive machine for a very long time. In those days, IBM 360s rented for tens of thousands of dollars per month.

So we told the operating system to use magnetic tape instead of the line printer. Our programs didn't care, because they had been written to use the IO abstractions of the operating system.

The 360 could pump out a full tape in 10 minutes or so—enough to print several rolls of form letters. The tapes were taken outside of the computer room and mounted on tape drives connected to offline printers. We had five of them, and we ran those five printers 24 hours per day, seven days per week, printing hundreds of thousands of pieces of junk mail every week.

The value of device independence was enormous! We could write our programs without knowing or caring which device would be used. We could test those programs using the local line printer connected to the computer. Then we could tell the operating system to "print" to magnetic tape and run off hundreds of thousands of forms.

Our programs had a shape. That shape disconnected policy from detail. The policy was the formatting of the name and address records. The detail was the device. We deferred the decision about which device we would use.

PHYSICAL ADDRESSING

In the early 1970s, I worked on a large accounting system for a local truckers union. We had a 25MB disk drive on which we stored records for `Agents`, `Employers`, and `Members`. The different records had different sizes, so we formatted the first few cylinders of the disk so that each sector was just the size of an `Agent` record. The next few cylinders were formatted to have sectors that fit the `Employer` records. The last few cylinders were formatted to fit the `Member` records.

We wrote our software to know the detailed structure of the disk. It knew that the disk had 200 cylinders and 10 heads, and that each cylinder had several dozen sectors per head. It knew which cylinders held the `Agents`, `Employers`, and `Members`.